

Replacing a bookkeeping Excel workbook with a **real SaaS product**

How I rebuilt a service-business bookkeeping workbook, formulas, macros, and all, into a modern web app with IRS-sourced tax calculations.

Industry Bookkeeping / financial SaaS

Work Full-stack development · Tax research

Scope Own product, ongoing

THE PROBLEM

The original tool was a well-built Excel workbook, but Excel has limits. Twelve monthly tabs had to be kept in sync by hand, and tax estimates were guesswork instead of IRS math.

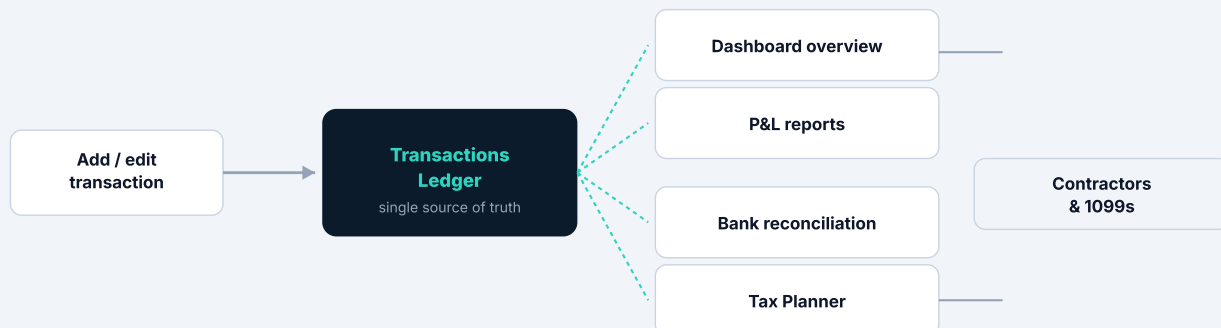
Twelve monthly tabs kept in sync with manual "Transfer" macros

Pivot tables that needed a manual refresh to stay accurate

An after-the-fact "Error, Double Entry" flag instead of real validation

Tax estimates based on guesswork, not actual IRS formulas

THE SYSTEM



WHAT I BUILT

01

Single-ledger architecture

One normalized transactions table instead of twelve monthly tabs, with category and type validation enforced before anything gets saved.

02

IRS-accurate Tax Planner

Self-employment tax, the QBI deduction, and true IRS safe-harbor quarterly payments, calculated from real bookkeeping data and versioned by tax year.

03

Contractors & 1099 tracking

Every contract-labor payment grouped by vendor, with the correct 1099-NEC threshold applied automatically for the current tax year.

04

Bank reconciliation & reporting

Real reconciliation against bank statements, plus Profit and Loss reports by month, quarter, or custom range.

THE RESULT

Twelve tabs become one

A single ledger replaces the monthly-tab system and its manual transfer macros entirely.

Tax figures you can trust

Every formula re-verified against primary IRS sources, catching stale-data bugs and a missing law change before they reached users.

Nothing slips through

1099 thresholds and safe-harbor payments calculate correctly on their own, year over year.

HOW I WORK

1

Reverse-engineer the source of truth

Fully documented the original workbook's formulas, macros, and business rules before writing a line of app code.

2

Build on real infrastructure

Next.js, Postgres, and proper server-side validation, not a prototype that only works in the happy path.

3

Verify against primary sources

Every tax calculation checked against actual IRS publications, not secondhand summaries.

4

Keep auditing after launch

Re-checked the entire tax engine when a real user asked if the numbers were accurate, which is how a law change got caught.

TOOLS I WORK WITH

Next.js

TypeScript

PostgreSQL

Drizzle ORM

NextAuth

Tailwind CSS

Recharts

Zod

Vercel

ALSO DONE FOR OTHER CLIENTS

Alongside CRM and marketing automation work for clients, I also design and ship my own products end to end, from architecture and data modeling to tax-law research and production deployment.

Need something built, not just connected?

Send me a message with what you're trying to build.

Scope call
1 session

MVP build
core features

Full product
end to end